

$$L_{\mathbf{w},b} \stackrel{\text{def}}{=} \prod_{i=1 \dots N} f_{\mathbf{w},b}(\mathbf{x}_i)^{y_i} (1 - f_{\mathbf{w},b}(\mathbf{x}_i))^{(1-y_i)}. \quad (3.4)$$

Выражение $f_{\mathbf{w},b}(\mathbf{x})^{y_i} (1 - f_{\mathbf{w},b}(\mathbf{x}))^{(1-y_i)}$ может показаться пугающим, но это всего лишь причудливый математический способ сказать: « $f_{\mathbf{w},b}(\mathbf{x})$, когда $y_i = 1$, и $(1 - f_{\mathbf{w},b}(\mathbf{x}))$ иначе». Действительно, если $y_i = 1$, тогда $(1 - f_{\mathbf{w},b}(\mathbf{x}))^{(1-y_i)}$ равно 1, потому что $(1 - y_i) = 0$, а как мы знаем, любое число в степени 0 равно 1. С другой стороны, если $y_i = 0$, тогда $f_{\mathbf{w},b}(\mathbf{x})^{y_i}$ равно 1 по той же причине.

Возможно, вы обратили внимание, что в целевой функции мы использовали оператор произведения Π вместо оператора суммы Σ , который применялся в линейной регрессии. Это связано с тем, что вероятность наблюдения N меток в N образцах является произведением вероятностей каждого наблюдения (при условии, что все наблюдения независимы друг от друга, что в нашем случае действительно так). Можно провести параллель с умножением вероятностей исходов в серии независимых экспериментов в теории вероятностей.

Поскольку в модели используется функция $\exp$, на практике удобнее максимизировать **логарифм правдоподобия**, а не правдоподобие. Логарифм правдоподобия определяется следующим образом:

$$\text{Log } L_{\mathbf{w},b} \stackrel{\text{def}}{=} \ln(L_{\mathbf{w},b}(\mathbf{x})) = \sum_{i=1}^N [y_i \ln f_{\mathbf{w},b}(\mathbf{x}) + (1 - y_i) \ln (1 - f_{\mathbf{w},b}(\mathbf{x}))].$$

Поскольку $\ln$ — **строго возрастающая функция**, ее максимизация равносильна максимизации ее аргумента, а решение этой новой задачи оптимизации равносильно решению исходной задачи.

В отличие от линейной регрессии, задача оптимизации выше не имеет аналитического решения. Поэтому в таких случаях обычно используется процедура численной оптимизации — **градиентный спуск**. Но об этом мы поговорим в следующей главе.

3.3. Обучение дерева решений

Дерево решений — это ациклический **граф**, который можно использовать для принятия решений. В каждом ветвящемся узле графа исследуется j -й признак из вектора признаков. Если значение признака ниже определенного порога, выбирается левая ветвь; иначе — правая. По достижении листового узла принимается решение о классе, к которому относится образец.

Как следует из заголовка раздела, дерево решений можно получить из данных.

3.3.1. Задача

Как и прежде, дано: коллекция размеченных данных; метки принадлежат множеству $\{0, 1\}$. Нужно построить дерево решений, которое позволило бы предсказать класс по заданному вектору признаков.

3.3.2. Решение

Существуют различные формулировки алгоритма обучения дерева решений. В этой книге мы рассмотрим только один, который называется **ID3**.

Критерием оптимизации в данном случае является среднее логарифмическое правдоподобие:

$$\frac{1}{N} \sum_{i=1}^N \left[y_i \ln f_{ID3}(\mathbf{x}_i) + (1 - y_i) \ln (1 - f_{ID3}(\mathbf{x}_i)) \right], \quad (3.5)$$

где f_{ID3} — дерево решений.

Пока все выглядит похожим на логистическую регрессию. Однако, в отличие от алгоритма обучения логистической регрессии, который строит **параметрическую модель** $f_{\mathbf{w}^*, b^*}$, путем поиска оптимального решения критерия оптимизации, алгоритм ID3 оптимизирует его *приблизительно*, конструируя **непараметрическую модель** $f_{ID3}^{\text{def}}(\mathbf{x}) = \Pr(y = 1 | \mathbf{x})$.

Алгоритм обучения ID3 работает следующим образом. Пусть $\mathcal{S}$ обозначает множество размеченных данных. В начале дерево решений имеет только начальный узел, который содержит все данные: $\mathcal{S} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$. Начнем с постоянной модели f_{ID3}^S , которая определяется как

$$f_{ID3}^S \stackrel{\text{def}}{=} \frac{1}{|\mathcal{S}|} \sum_{(\mathbf{x}, y) \in \mathcal{S}} y. \quad (3.6)$$

Вышеупомянутая модель $f_{ID3}^S(\mathbf{x})$ будет возвращать один и тот же прогноз для любого входа $\mathbf{x}$. Соответствующее дерево решений, построенное на основе нашего фиктивного набора данных из 12 размеченных данных, показано на рис. 3.4а.

Затем мы ищем все признаки $j = 1, \dots, D$ и все пороги t и разбиваем множество $\mathcal{S}$ на два подмножества: $\mathcal{S}_- \stackrel{\text{def}}{=} \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{S}, x^{(j)} < t\}$ и $\mathcal{S}_+ \stackrel{\text{def}}{=} \{(\mathbf{x}, y) | (\mathbf{x}, y) \in \mathcal{S}, x^{(j)} \geq t\}$. Два новых подмножества образуют два новых листовых узла, и мы оцениваем для

всех возможных пар (j, t) , насколько хорошим получилось расщепление на части $\mathcal{S}_-$ и $\mathcal{S}_+$. Наконец, выбираем наилучшие значения (j, t) , разбиваем $\mathcal{S}$ на $\mathcal{S}_-$ и $\mathcal{S}_+$, формируем два новых листовых узла и продолжаем рекурсивно разбивать на $\mathcal{S}_-$ и $\mathcal{S}_+$ (или завершаем работу, если ни одно разбиение не дает модели, которая позволяет получить лучший прогноз, чем текущая). Дерево решений после одного расщепления показано на рис. 3.4б.

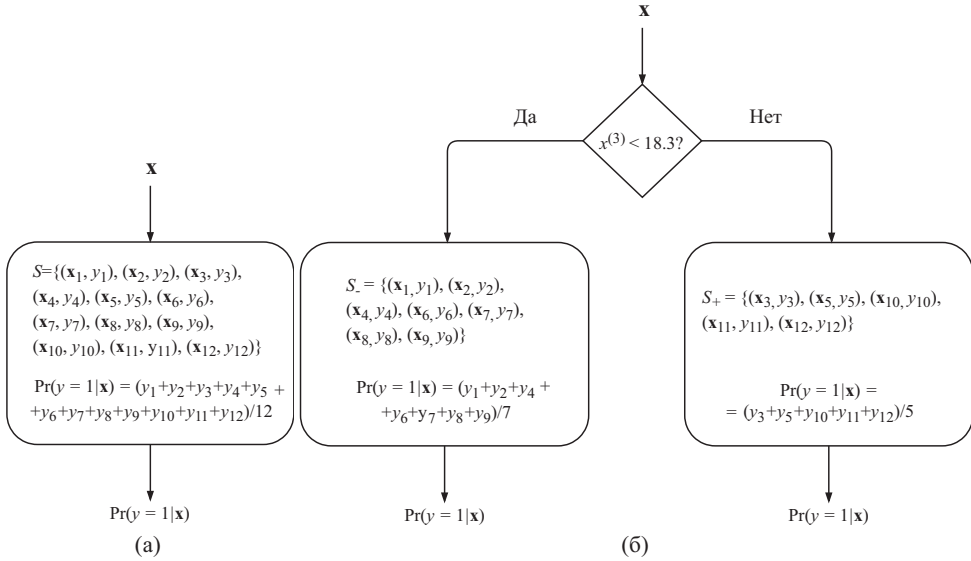


Рис. 3.4. Иллюстрация алгоритма построения дерева решений. Множество $\mathcal{S}$ содержит 12 размеченных данных. (а) Вначале дерево решений содержит только начальный узел; он дает один и тот же прогноз для любого входа. (б) Дерево решений после первого расщепления; оно проверяет, является ли признак 3 меньше 18.3, и, в зависимости от результата, прогноз выполняется одним из двух листовых узлов

Теперь у вас наверняка возник вопрос, что означают слова «оценить, насколько хорошим получилось расщепление». В ID3 качество расщепления оценивается с использованием критерия, называемого **энтропией**. Энтропия — это мера неопределенности случайной величины. Она достигает своего максимума, когда все значения случайной величины равновероятны. Энтропия достигает своего минимума, когда случайная величина может иметь только одно значение. Энтропия множества данных $\mathcal{S}$ определяется как

$$H(\mathcal{S}) \stackrel{\text{def}}{=} -f_{ID3}^{\mathcal{S}} \ln f_{ID3}^{\mathcal{S}} - (1 - f_{ID3}^{\mathcal{S}}) \ln (1 - f_{ID3}^{\mathcal{S}}).$$

Когда мы разбиваем множество данных по некоторому признаку j и порогу t , энтропия разбиения $H(\mathcal{S}_-, \mathcal{S}_+)$ определяется как простая взвешенная сумма двух энтропий:

$$H(\mathcal{S}_-, \mathcal{S}_+) \stackrel{\text{def}}{=} \frac{|\mathcal{S}_-|}{|\mathcal{S}|} H(\mathcal{S}_-) + \frac{|\mathcal{S}_+|}{|\mathcal{S}|} H(\mathcal{S}_+). \quad (3.7)$$

Итак, в ID3, на каждом шаге, в каждом листовом узле мы находим расщепление, минимизирующее энтропию, заданную уравнением 3.7, или останавливаемся на этом листовом узле.

Алгоритм останавливается на листовом узле в любой из следующих ситуаций:

- Все примеры в листовом узле правильно классифицируются моделью (уравнение 3.6).
- Невозможно найти атрибут для расщепления.
- Расщепление уменьшает энтропию ниже некоторого значения ϵ (которое нужно определить экспериментально¹).
- Дерево достигает некоторой максимальной глубины d (также должна определяться экспериментально).

Поскольку в ID3 решение о расщеплении набора данных в каждой итерации является локальным (не зависит от будущих расщеплений), алгоритм не гарантирует оптимального решения. Модель можно улучшить, используя в процессе поиска оптимального дерева решений такие методы, как *возврат* (backtracking), хотя и за счет увеличения времени построения модели.

Наиболее широко используемая версия алгоритма обучения дерева решений называется **С4.5**. Она имеет несколько дополнительных особенностей по сравнению с ID3:

- принимает непрерывные и дискретные признаки;
- поддерживает возможность обработки неполных данных;
- решает проблему переобучения с использованием восходящего метода, известного как «подрезка» (отсечение ветвей).

Подрезка заключается в том, чтобы выполнить обратный обход только что созданного дерева и удалить ветви, которые не вносят существенного вклада в уменьшение ошибки, заменив их листовыми узлами.

¹ Я покажу, как это сделать, в главе 5, в разделе, посвященном настройке гиперпараметров.

In Figure 3.17, we can now see that the decision boundary around the classes 0 and 1 is much tighter using a relatively large value of γ :

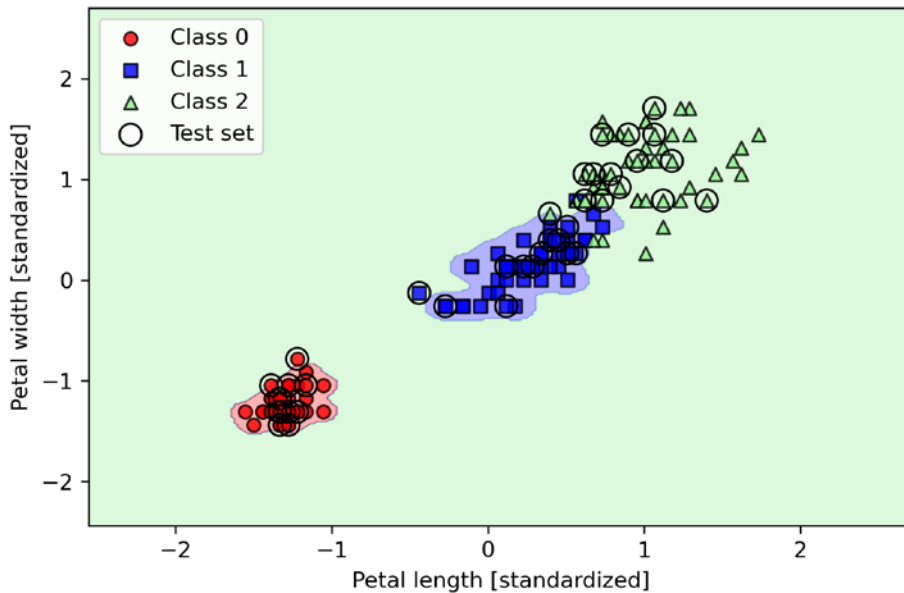


Figure 3.17: The decision boundaries on the Iris dataset using an RBF kernel SVM model with a large γ value

Although the model fits the training dataset very well, such a classifier will likely have a high generalization error on unseen data. This illustrates that the γ parameter also plays an important role in controlling overfitting or variance when the algorithm is too sensitive to fluctuations in the training dataset.

Decision tree learning

Decision tree classifiers are attractive models if we care about interpretability. As the name “decision tree” suggests, we can think of this model as breaking down our data by making a decision based on asking a series of questions.

Let's consider the following example in which we use a decision tree to decide upon an activity on a particular day:

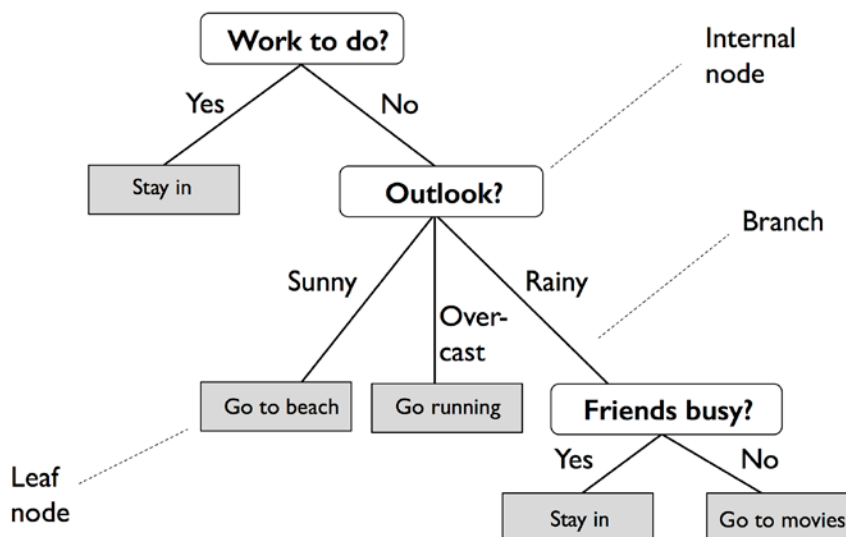


Figure 3.18: An example of a decision tree

Based on the features in our training dataset, the decision tree model learns a series of questions to infer the class labels of the examples. Although Figure 3.18 illustrates the concept of a decision tree based on categorical variables, the same concept applies if our features are real numbers, like in the Iris dataset. For example, we could simply define a cut-off value along the **sepal width** feature axis and ask a binary question: "Is the sepal width ≥ 2.8 cm?"

Using the decision algorithm, we start at the tree root and split the data on the feature that results in the largest **information gain (IG)**, which will be explained in more detail in the following section. In an iterative process, we can then repeat this splitting procedure at each child node until the leaves are pure. This means that the training examples at each node all belong to the same class. In practice, this can result in a very deep tree with many nodes, which can easily lead to overfitting. Thus, we typically want to **prune** the tree by setting a limit for the maximum depth of the tree.

Maximizing IG – getting the most bang for your buck

To split the nodes at the most informative features, we need to define an objective function to optimize via the tree learning algorithm. Here, our objective function is to maximize the IG at each split, which we define as follows:

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^m \frac{N_j}{N_p} I(D_j)$$

Here, f is the feature to perform the split; D_p and D_j are the dataset of the parent and j th child node; I is our **impurity** measure; N_p is the total number of training examples at the parent node; and N_j is the number of examples in the j th child node. As we can see, the information gain is simply the difference between the impurity of the parent node and the sum of the child node impurities—the lower the impurities of the child nodes, the larger the information gain. However, for simplicity and to reduce the combinatorial search space, most libraries (including scikit-learn) implement binary decision trees. This means that each parent node is split into two child nodes, D_{left} and D_{right} :

$$IG(D_p, f) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right})$$

The three impurity measures or splitting criteria that are commonly used in binary decision trees are **Gini impurity** (I_G), **entropy** (I_H), and the **classification error** (I_E). Let's start with the definition of entropy for all **non-empty** classes ($p(i|t) \neq 0$):

$$I_H(t) = - \sum_{i=1}^c p(i|t) \log_2 p(i|t)$$

Here, $p(i|t)$ is the proportion of the examples that belong to class i for a particular node, t . The entropy is therefore 0 if all examples at a node belong to the same class, and the entropy is maximal if we have a uniform class distribution. For example, in a binary class setting, the entropy is 0 if $p(i=1|t) = 1$ or $p(i=0|t) = 0$. If the classes are distributed uniformly with $p(i=1|t) = 0.5$ and $p(i=0|t) = 0.5$, the entropy is 1. Therefore, we can say that the entropy criterion attempts to maximize the mutual information in the tree.

To provide a visual intuition, let us visualize the entropy values for different class distributions via the following code:

```
>>> def entropy(p):
...     return - p * np.log2(p) - (1 - p) * np.log2((1 - p))
>>> x = np.arange(0.0, 1.0, 0.01)
>>> ent = [entropy(p) if p != 0 else None for p in x]
>>> plt.ylabel('Entropy')
>>> plt.xlabel('Class-membership probability p(i=1)')
>>> plt.plot(x, ent)
>>> plt.show()
```

Figure 3.19 below shows the plot produced by the preceding code:

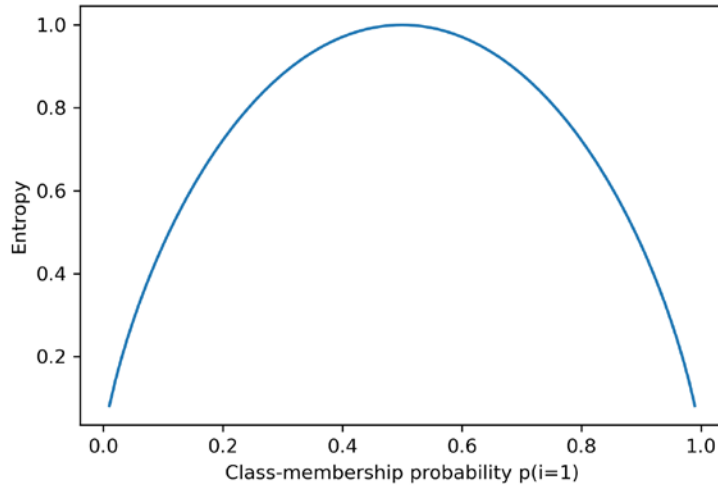


Figure 3.19: Entropy values for different class-membership probabilities

The Gini impurity can be understood as a criterion to minimize the probability of misclassification:

$$I_G(t) = \sum_{i=1}^c p(i|t) (1 - p(i|t)) = 1 - \sum_{i=1}^c p(i|t)^2$$

Similar to entropy, the Gini impurity is maximal if the classes are perfectly mixed, for example, in a binary class setting ($c = 2$):

$$I_G(t) = 1 - \sum_{i=1}^c 0.5^2 = 0.5$$

However, in practice, both the Gini impurity and entropy typically yield very similar results, and it is often not worth spending much time on evaluating trees using different impurity criteria rather than experimenting with different pruning cut-offs. In fact, as you will see later in Figure 3.21, both the Gini impurity and entropy have a similar shape.

Another impurity measure is the classification error:

$$I_E(t) = 1 - \max\{p(i|t)\}$$

This is a useful criterion for pruning, but not recommended for growing a decision tree, since it is less sensitive to changes in the class probabilities of the nodes. We can illustrate this by looking at the two possible splitting scenarios shown in Figure 3.20:



Figure 3.20: Decision tree data splits

We start with a dataset, D_p , at the parent node, which consists of 40 examples from class 1 and 40 examples from class 2 that we split into two datasets, D_{left} and D_{right} . The information gain using the classification error as a splitting criterion would be the same ($IG_E = 0.25$) in both scenarios, A and B:

$$I_E(D_p) = 1 - 0.5 = 0.5$$

$$A: \quad I_E(D_{left}) = 1 - \frac{3}{4} = 0.25$$

$$A: \quad I_E(D_{right}) = 1 - \frac{3}{4} = 0.25$$

$$A: \quad IG_E = 0.5 - \frac{4}{8}0.25 - \frac{4}{8}0.25 = 0.25$$

$$B: \quad I_E(D_{left}) = 1 - \frac{4}{6} = \frac{1}{3}$$

$$B: \quad I_E(D_{right}) = 1 - 1 = 0$$

$$B: \quad IG_E = 0.5 - \frac{6}{8} \times \frac{1}{3} - 0 = 0.25$$

However, the Gini impurity would favor the split in scenario B ($IG_G = 0.1\bar{6}$) over scenario A ($IG_G = 0.125$), which is indeed purer:

$$I_G(D_p) = 1 - (0.5^2 + 0.5^2) = 0.5$$

$$A: \quad I_G(D_{left}) = 1 - \left(\left(\frac{3}{4} \right)^2 + \left(\frac{1}{4} \right)^2 \right) = \frac{3}{8} = 0.375$$

$$A: \quad I_G(D_{right}) = 1 - \left(\left(\frac{1}{4} \right)^2 + \left(\frac{3}{4} \right)^2 \right) = \frac{3}{8} = 0.375$$

$$A: \quad IG_G = 0.5 - \frac{4}{8}0.375 - \frac{4}{8}0.375 = 0.125$$

$$B: \quad I_G(D_{left}) = 1 - \left(\left(\frac{2}{6} \right)^2 + \left(\frac{4}{6} \right)^2 \right) = \frac{4}{9} = 0.\bar{4}$$

$$B: \quad I_G(D_{right}) = 1 - (1^2 + 0^2) = 0$$

$$B: \quad IG_G = 0.5 - \frac{6}{8}0.\bar{4} - 0 = 0.1\bar{6}$$

Similarly, the entropy criterion would also favor scenario B ($IG_H = 0.31$) over scenario A ($IG_H = 0.19$):

$$I_H(D_p) = -(0.5 \log_2(0.5) + 0.5 \log_2(0.5)) = 1$$

$$A: \quad I_H(D_{left}) = -\left(\frac{3}{4} \log_2\left(\frac{3}{4}\right) + \frac{1}{4} \log_2\left(\frac{1}{4}\right)\right) = 0.81$$

$$A: \quad I_H(D_{right}) = -\left(\frac{1}{4} \log_2\left(\frac{1}{4}\right) + \frac{3}{4} \log_2\left(\frac{3}{4}\right)\right) = 0.81$$

$$A: \quad IG_H = 1 - \frac{4}{8} 0.81 - \frac{4}{8} 0.81 = 0.19$$

$$B: \quad I_H(D_{left}) = -\left(\frac{2}{6} \log_2\left(\frac{2}{6}\right) + \frac{4}{6} \log_2\left(\frac{4}{6}\right)\right) = 0.92$$

$$B: \quad I_H(D_{right}) = 0$$

$$B: \quad IG_H = 1 - \frac{6}{8} 0.92 - 0 = 0.31$$

For a more visual comparison of the three different impurity criteria that we discussed previously, let's plot the impurity indices for the probability range $[0, 1]$ for class 1. Note that we will also add a scaled version of the entropy (entropy / 2) to observe that the Gini impurity is an intermediate measure between entropy and the classification error. The code is as follows:

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> def gini(p):
...     return p*(1 - p) + (1 - p)*(1 - (1-p))
>>> def entropy(p):
...     return - p*np.log2(p) - (1 - p)*np.log2((1 - p))
>>> def error(p):
...     return 1 - np.max([p, 1 - p])
>>> x = np.arange(0.0, 1.0, 0.01)
>>> ent = [entropy(p) if p != 0 else None for p in x]
>>> sc_ent = [e*0.5 if e else None for e in ent]
>>> err = [error(i) for i in x]
>>> fig = plt.figure()
>>> ax = plt.subplot(111)
```

```

>>> for i, lab, ls, c, in zip([ent, sc_ent, gini(x), err],
...                           ['Entropy', 'Entropy (scaled)',
...                           'Gini impurity',
...                           'Misclassification error'],
...                           ['- ', '- ', '-- ', '-. '],
...                           ['black', 'lightgray',
...                           'red', 'green', 'cyan']):
...     line = ax.plot(x, i, label=lab,
...                    linestyle=ls, lw=2, color=c)
>>> ax.legend(loc='upper center', bbox_to_anchor=(0.5, 1.15),
...           ncol=5, fancybox=True, shadow=False)
>>> ax.axhline(y=0.5, linewidth=1, color='k', linestyle='--')
>>> ax.axhline(y=1.0, linewidth=1, color='k', linestyle='--')
>>> plt.ylim([0, 1.1])
>>> plt.xlabel('p(i=1)')
>>> plt.ylabel('impurity index')
>>> plt.show()

```

The plot produced by the preceding code example is as follows:

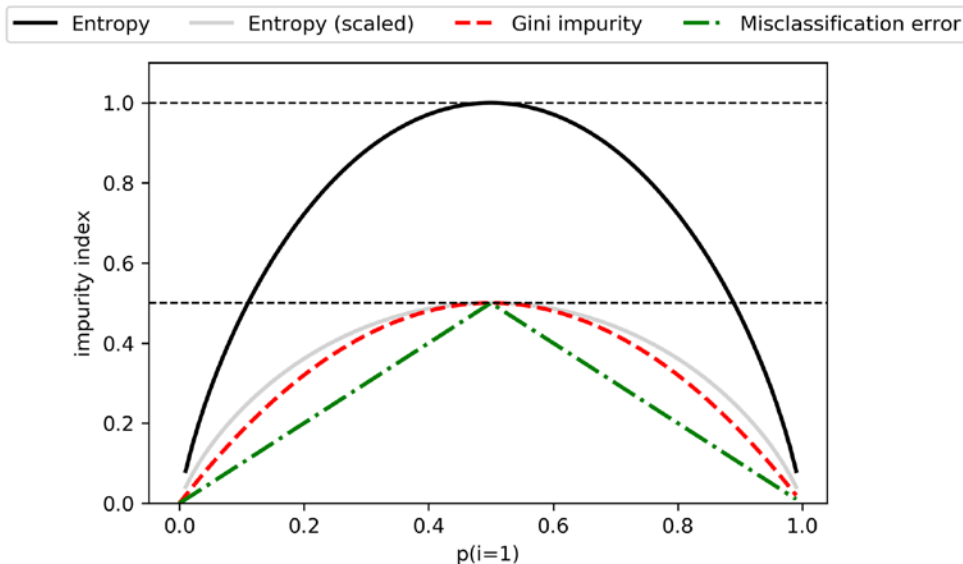


Figure 3.21: The different impurity indices for different class-membership probabilities between 0 and 1

Building a decision tree

Decision trees can build complex decision boundaries by dividing the feature space into rectangles. However, we have to be careful since the deeper the decision tree, the more complex the decision boundary becomes, which can easily result in overfitting. Using scikit-learn, we will now train a decision tree with a maximum depth of 4, using the Gini impurity as a criterion for impurity.

Although feature scaling may be desired for visualization purposes, note that feature scaling is not a requirement for decision tree algorithms. The code is as follows:

```
>>> from sklearn.tree import DecisionTreeClassifier
>>> tree_model = DecisionTreeClassifier(criterion='gini',
...                                   max_depth=4,
...                                   random_state=1)
>>> tree_model.fit(X_train, y_train)
>>> X_combined = np.vstack((X_train, X_test))
>>> y_combined = np.hstack((y_train, y_test))
>>> plot_decision_regions(X_combined,
...                       y_combined,
...                       classifier=tree_model,
...                       test_idx=range(105, 150))
>>> plt.xlabel('Petal length [cm]')
>>> plt.ylabel('Petal width [cm]')
>>> plt.legend(loc='upper left')
>>> plt.tight_layout()
>>> plt.show()
```

After executing the code example, we get the typical axis-parallel decision boundaries of the decision tree:

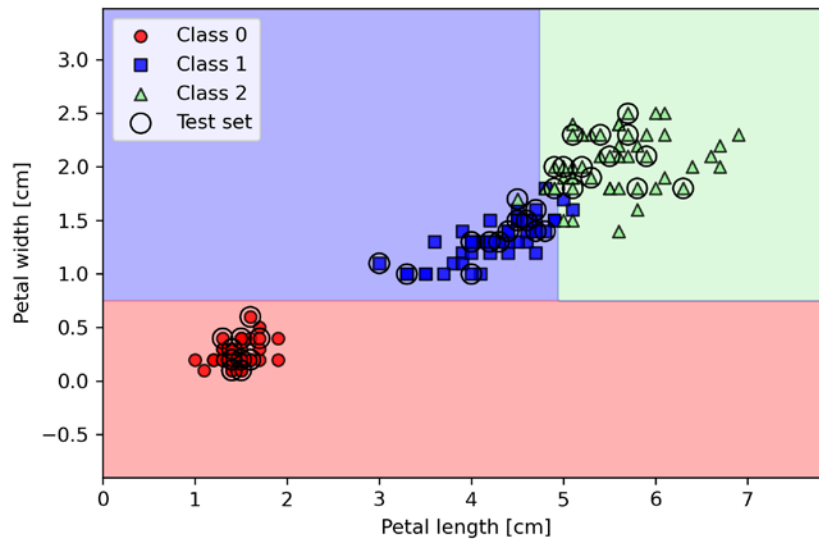


Figure 3.22: The decision boundaries of the Iris data using a decision tree

A nice feature in scikit-learn is that it allows us to readily visualize the decision tree model after training via the following code:

```
>>> from sklearn import tree
>>> feature_names = ['Sepal length', 'Sepal width',
...                  'Petal length', 'Petal width']
>>> tree.plot_tree(tree_model,
...                 feature_names=feature_names,
...                 filled=True)
>>> plt.show()
```

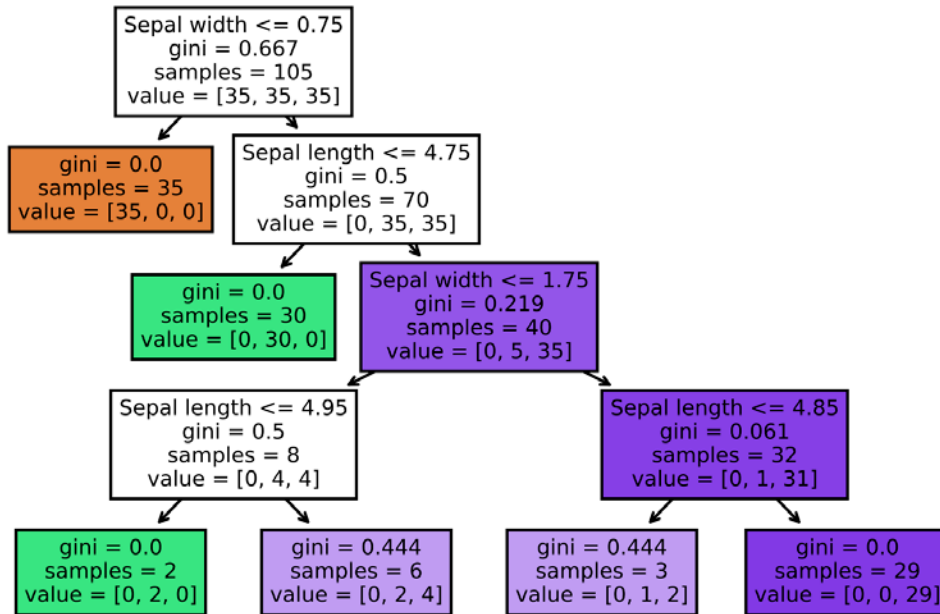


Figure 3.23: A decision tree model fit to the Iris dataset

Setting `filled=True` in the `plot_tree` function we called colors the nodes by the majority class label at that node. There are many additional options available, which you can find in the documentation at https://scikit-learn.org/stable/modules/generated/sklearn.tree.plot_tree.html.

Looking at the decision tree figure, we can now nicely trace back the splits that the decision tree determined from our training dataset. Regarding the feature splitting criterion at each node, note that the branches to the left correspond to “True” and branches to the right correspond to “False.”

Looking at the root node, it starts with 105 examples at the top. The first split uses a sepal width cut-off ≤ 0.75 cm for splitting the root node into two child nodes with 35 examples (left child node) and 70 examples (right child node). After the first split, we can see that the left child node is already pure and only contains examples from the *Iris-setosa* class (Gini impurity = 0). The further splits on the right are then used to separate the examples from the *Iris-versicolor* and *Iris-virginica* class.

Looking at this tree, and the decision region plot of the tree, we can see that the decision tree does a very good job of separating the flower classes. Unfortunately, scikit-learn currently does not implement functionality to manually post-prune a decision tree. However, we could go back to our previous code example, change the `max_depth` of our decision tree to 3, and compare it to our current model, but we leave this as an exercise for the interested reader.

Alternatively, scikit-learn provides an automatic cost complexity post-pruning procedure for decision trees. Interested readers can find more information about this more advanced topic in the following tutorial: https://scikit-learn.org/stable/auto_examples/tree/plot_cost_complexity_pruning.html.

Combining multiple decision trees via random forests

Ensemble methods have gained huge popularity in applications of machine learning during the last decade due to their good classification performance and robustness toward overfitting. While we are going to cover different ensemble methods, including **bagging** and **boosting**, later in *Chapter 7, Combining Different Models for Ensemble Learning*, let's discuss the decision tree-based **random forest** algorithm, which is known for its good scalability and ease of use. A random forest can be considered as an **ensemble** of decision trees. The idea behind a random forest is to average multiple (deep) decision trees that individually suffer from high variance to build a more robust model that has a better generalization performance and is less susceptible to overfitting. The random forest algorithm can be summarized in four simple steps:

1. Draw a random **bootstrap** sample of size n (randomly choose n examples from the training dataset with replacement).
2. Grow a decision tree from the bootstrap sample. At each node:
 - a. Randomly select d features without replacement.
 - b. Split the node using the feature that provides the best split according to the objective function, for instance, maximizing the information gain.
3. Repeat *steps 1-2* k times.
4. Aggregate the prediction by each tree to assign the class label by **majority vote**. Majority voting will be discussed in more detail in *Chapter 7*.

We should note one slight modification in *step 2* when we are training the individual decision trees: instead of evaluating all features to determine the best split at each node, we only consider a random subset of those.

Sampling with and without replacement



In case you are not familiar with the terms sampling “with” and “without” replacement, let’s walk through a simple thought experiment. Let’s assume that we are playing a lottery game where we randomly draw numbers from an urn. We start with an urn that holds five unique numbers, 0, 1, 2, 3, and 4, and we draw exactly one number on each turn. In the first round, the chance of drawing a particular number from the urn would be $1/5$. Now, in sampling without replacement, we do not put the number back into the urn after each turn. Consequently, the probability of drawing a particular number from the set of remaining numbers in the next round depends on the previous round. For example, if we have a remaining set of numbers 0, 1, 2, and 4, the chance of drawing number 0 would become $1/4$ in the next turn.

However, in random sampling with replacement, we always return the drawn number to the urn so that the probability of drawing a particular number at each turn does not change; we can draw the same number more than once. In other words, in sampling *with* replacement, the samples (numbers) are independent and have a covariance of zero. For example, the results from five rounds of drawing random numbers could look like this:

- Random sampling without replacement: 2, 1, 3, 4, 0
- Random sampling with replacement: 1, 3, 3, 4, 1

Although random forests don’t offer the same level of interpretability as decision trees, a big advantage of random forests is that we don’t have to worry so much about choosing good hyperparameter values. We typically don’t need to prune the random forest since the ensemble model is quite robust to noise from averaging the predictions among the individual decision trees. The only parameter that we need to care about in practice is the number of trees, k , (*step 3*) that we choose for the random forest. Typically, the larger the number of trees, the better the performance of the random forest classifier at the expense of an increased computational cost.

Although it is less common in practice, other hyperparameters of the random forest classifier that can be optimized—using techniques that we will discuss in *Chapter 6, Learning Best Practices for Model Evaluation and Hyperparameter Tuning*—are the size, n , of the bootstrap sample (*step 1*) and the number of features, d , that are randomly chosen for each split (*step 2a*), respectively. Via the sample size, n , of the bootstrap sample, we control the bias-variance tradeoff of the random forest.

Decreasing the size of the bootstrap sample increases the diversity among the individual trees since the probability that a particular training example is included in the bootstrap sample is lower. Thus, shrinking the size of the bootstrap samples may increase the *randomness* of the random forest, and it can help to reduce the effect of overfitting. However, smaller bootstrap samples typically result in a lower overall performance of the random forest and a small gap between training and test performance, but a low test performance overall. Conversely, increasing the size of the bootstrap sample may increase the degree of overfitting. Because the bootstrap samples, and consequently the individual decision trees, become more similar to one another, they learn to fit the original training dataset more closely.

In most implementations, including the `RandomForestClassifier` implementation in scikit-learn, the size of the bootstrap sample is chosen to be equal to the number of training examples in the original training dataset, which usually provides a good bias-variance tradeoff. For the number of features, d , at each split, we want to choose a value that is smaller than the total number of features in the training dataset. A reasonable default that is used in scikit-learn and other implementations is $d = \sqrt{m}$, where m is the number of features in the training dataset.

Conveniently, we don't have to construct the random forest classifier from individual decision trees by ourselves because there is already an implementation in scikit-learn that we can use:

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> forest = RandomForestClassifier(n_estimators=25,
...                               random_state=1,
...                               n_jobs=2)
>>> forest.fit(X_train, y_train)
>>> plot_decision_regions(X_combined, y_combined,
...                       classifier=forest, test_idx=range(105,150))
>>> plt.xlabel('Petal length [cm]')
>>> plt.ylabel('Petal width [cm]')
>>> plt.legend(loc='upper left')
>>> plt.tight_layout()
>>> plt.show()
```

As we can see, using quadratic or cubic features does not really have an effect. That's because the relationship between the two variables appears to be linear. So, let's take a look at another feature, namely, Overall Qual. The Overall Qual variable rates the overall quality of the material and finish of the houses and is given on a scale from 1 to 10, where 10 is best:

```
>>> X = df[['Overall Qual']].values
>>> y = df['SalePrice'].values
```

After specifying the X and y variables, we can reuse the previous code and obtain the plot in Figure 9.14:

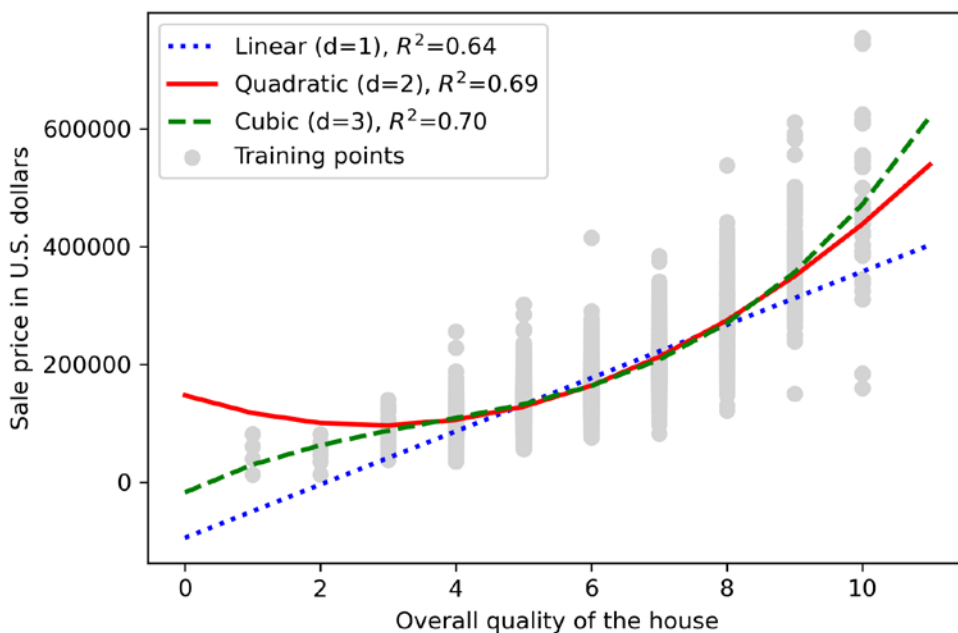


Figure 9.14: A linear, quadratic, and cubic fit on the sale price and house quality data

As you can see, the quadratic and cubic fits capture the relationship between sale prices and the overall quality of the house better than the linear fit. However, you should be aware that adding more and more polynomial features increases the complexity of a model and therefore increases the chance of overfitting. Thus, in practice, it is always recommended to evaluate the performance of the model on a separate test dataset to estimate the generalization performance.

Dealing with nonlinear relationships using random forests

In this section, we are going to look at **random forest** regression, which is conceptually different from the previous regression models in this chapter. A random forest, which is an ensemble of multiple **decision trees**, can be understood as the sum of piecewise linear functions, in contrast to the global linear and polynomial regression models that we discussed previously. In other words, via the decision tree algorithm, we subdivide the input space into smaller regions that become more manageable.

Decision tree regression

An advantage of the decision tree algorithm is that it works with arbitrary features and does not require any transformation of the features if we are dealing with nonlinear data because decision trees analyze one feature at a time, rather than taking weighted combinations into account. (Likewise, normalizing or standardizing features is not required for decision trees.) As mentioned in *Chapter 3, A Tour of Machine Learning Classifiers Using Scikit-Learn*, we grow a decision tree by iteratively splitting its nodes until the leaves are pure or a stopping criterion is satisfied. When we used decision trees for classification, we defined entropy as a measure of impurity to determine which feature split maximizes the **information gain (IG)**, which can be defined as follows for a binary split:

$$IG(D_p, x_i) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right})$$

Here, x_i is the feature to perform the split, N_p is the number of training examples in the parent node, I is the impurity function, D_p is the subset of training examples at the parent node, and D_{left} and D_{right} are the subsets of training examples at the left and right child nodes after the split. Remember that our goal is to find the feature split that maximizes the information gain; in other words, we want to find the feature split that reduces the impurities in the child nodes most. In *Chapter 3*, we discussed Gini impurity and entropy as measures of impurity, which are both useful criteria for classification. To use a decision tree for regression, however, we need an impurity metric that is suitable for continuous variables, so we define the impurity measure of a node, t , as the MSE instead:

$$I(t) = MSE(t) = \frac{1}{N_t} \sum_{i \in D_t} (y^{(i)} - \hat{y}_t)^2$$

Here, N_t is the number of training examples at node t , D_t is the training subset at node t , $y^{(i)}$ is the true target value, and $\hat{y}_t$ is the predicted target value (sample mean):

$$\hat{y}_t = \frac{1}{N_t} \sum_{i \in D_t} y^{(i)}$$

In the context of decision tree regression, the MSE is often referred to as **within-node variance**, which is why the splitting criterion is also better known as **variance reduction**.

To see what the line fit of a decision tree looks like, let's use the `DecisionTreeRegressor` implemented in scikit-learn to model the relationship between the `SalePrice` and `Gr_Living_Area` variables. Note that `SalePrice` and `Gr_Living_Area` do not necessarily represent a nonlinear relationship, but this feature combination still demonstrates the general aspects of a regression tree quite nicely:

```
>>> from sklearn.tree import DecisionTreeRegressor
>>> X = df[['Gr_Liv_Area']].values
>>> y = df['SalePrice'].values
>>> tree = DecisionTreeRegressor(max_depth=3)
>>> tree.fit(X, y)
```

```
>>> sort_idx = X.flatten().argsort()
>>> lin_regplot(X[sort_idx], y[sort_idx], tree)
>>> plt.xlabel('Living area above ground in square feet')
>>> plt.ylabel('Sale price in U.S. dollars')>>> plt.show()
```

As you can see in the resulting plot, the decision tree captures the general trend in the data. And we can imagine that a regression tree could also capture trends in nonlinear data relatively well. However, a limitation of this model is that it does not capture the continuity and differentiability of the desired prediction. In addition, we need to be careful about choosing an appropriate value for the depth of the tree so as to not overfit or underfit the data; here, a depth of three seemed to be a good choice.

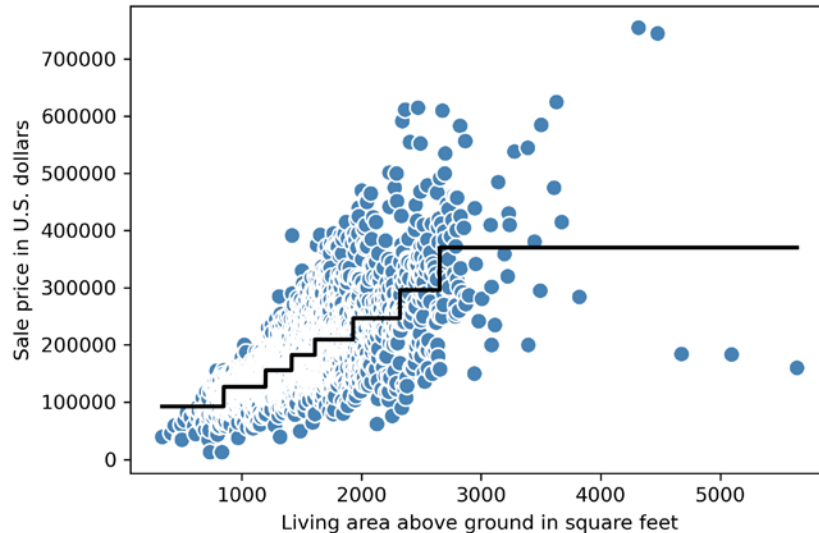


Figure 9.15: A decision tree regression plot

You are encouraged to experiment with deeper decision trees. Note that the relationship between `Gr Living Area` and `SalePrice` is rather linear, so you are also encouraged to apply the decision tree to the `Overall Qual` variable instead.

In the next section, we will look at a more robust way of fitting regression trees: random forests.

Random forest regression

As you learned in *Chapter 3*, the random forest algorithm is an ensemble technique that combines multiple decision trees. A random forest usually has a better generalization performance than an individual decision tree due to randomness, which helps to decrease the model's variance. Other advantages of random forests are that they are less sensitive to outliers in the dataset and don't require much parameter tuning. The only parameter in random forests that we typically need to experiment with is the number of trees in the ensemble. The basic random forest algorithm for regression is almost identical to the random forest algorithm for classification that we discussed in *Chapter 3*. The only difference is that we use the MSE criterion to grow the individual decision trees, and the predicted target variable is calculated as the average prediction across all decision trees.

Now, let's use all the features in the Ames Housing dataset to fit a random forest regression model on 70 percent of the examples and evaluate its performance on the remaining 30 percent, as we have done previously in the *Evaluating the performance of linear regression models* section. The code is as follows:

```
>>> target = 'SalePrice'
>>> features = df.columns[df.columns != target]
>>> X = df[features].values
>>> y = df[target].values
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.3, random_state=123)

>>> from sklearn.ensemble import RandomForestRegressor
>>> forest = RandomForestRegressor(
...     n_estimators=1000,
...     criterion='squared_error',
...     random_state=1,
...     n_jobs=-1)
>>> forest.fit(X_train, y_train)
>>> y_train_pred = forest.predict(X_train)
>>> y_test_pred = forest.predict(X_test)
>>> mae_train = mean_absolute_error(y_train, y_train_pred)
>>> mae_test = mean_absolute_error(y_test, y_test_pred)
>>> print(f'MAE train: {mae_train:.2f}')
MAE train: 8305.18
>>> print(f'MAE test: {mae_test:.2f}')
MAE test: 20821.77
>>> r2_train = r2_score(y_train, y_train_pred)
>>> r2_test = r2_score(y_test, y_test_pred)
>>> print(f'R^2 train: {r2_train:.2f}')
R^2 train: 0.98
>>> print(f'R^2 test: {r2_test:.2f}')
R^2 test: 0.85
```

Unfortunately, you can see that the random forest tends to overfit the training data. However, it's still able to explain the relationship between the target and explanatory variables relatively well ($R^2 = 0.85$ on the test dataset). For comparison, the linear model from the previous section, *Evaluating the performance of linear regression models*, which was fit to the same dataset, was overfitting less but performed worse on the test set ($R^2 = 0.75$).

Lastly, let's also take a look at the residuals of the prediction:

```
>>> x_max = np.max([np.max(y_train_pred), np.max(y_test_pred)])
>>> x_min = np.min([np.min(y_train_pred), np.min(y_test_pred)])

>>> fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7, 3), sharey=True)
```

```

>>> ax1.scatter(y_test_pred, y_test_pred - y_test,
...             c='limegreen', marker='s', edgecolor='white',
...             label='Test data')
>>> ax2.scatter(y_train_pred, y_train_pred - y_train,
...             c='steelblue', marker='o', edgecolor='white',
...             label='Training data')
>>> ax1.set_ylabel('Residuals')

>>> for ax in (ax1, ax2):
...     ax.set_xlabel('Predicted values')
...     ax.legend(loc='upper left')
...     ax.hlines(y=0, xmin=x_min-100, xmax=x_max+100,
...               color='black', lw=2)

>>> plt.tight_layout()
>>> plt.show()

```

As it was already summarized by the R^2 coefficient, you can see that the model fits the training data better than the test data, as indicated by the outliers in the y axis direction. Also, the distribution of the residuals does not seem to be completely random around the zero center point, indicating that the model is not able to capture all the exploratory information. However, the residual plot indicates a large improvement over the residual plot of the linear model that we plotted earlier in this chapter.

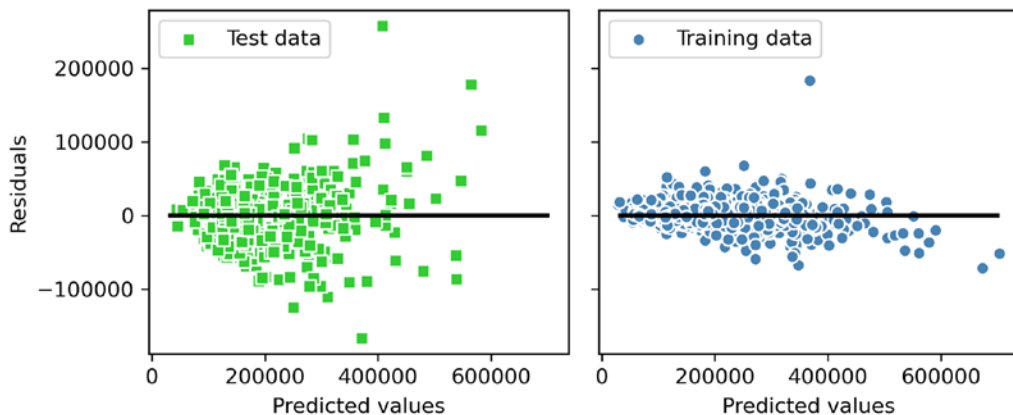


Figure 9.16: The residuals of the random forest regression